Towards Understanding Unsafe Video Generation

Yan Pang†, Aiping Xiong§, Yang Zhang‡, and Tianhao Wang†

†University of Virginia § Penn State University ‡CISPA Helmholtz Center for Information Security
{yanpang, tianhao}@virginia.edu, axx29@psu.edu, zhang@cispa.de

arXiv:2407.12581v1 [cs.CR] 17 Jul 2024

***Abstract*—Video generation models (VGMs) have demonstrated the capability to synthesize high-quality output. It is important to understand their potential to produce unsafe content, such as violent or terrifying videos. In this work, we provide a comprehensive understanding of unsafe video generation.**

First, to confirm the possibility that these models could indeed generate unsafe videos, we choose unsafe content generation prompts collected from 4chan and Lexica, and three open-source SOTA VGMs to generate unsafe videos. After filtering out duplicates and poorly generated content, we created an initial set of 2112 unsafe videos from an original pool of 5607 videos. Through clustering and thematic coding analysis of these generated videos, we identify 5 unsafe video categories: *Distorted/Weird*, *Terrifying*, *Pornographic*, *Violent/Bloody*, and *Political*. With IRB approval, we then recruit online participants to help label the generated videos. Based on the annotations submitted by 403 participants, we identified 937 unsafe videos from the initial video set. With the labeled information and the corresponding prompts, we created the first dataset of unsafe videos generated by VGMs.

We then study possible defense mechanisms to prevent the generation of unsafe videos. Existing defense methods in image generation focus on filtering either input prompt or output results. We propose a new approach called Latent Variable Defense (LVD), which works within the models internal sampling process. LVD can achieve 0.90 defense accuracy while reducing time and computing resources by $10\times$ when sampling a large number of unsafe prompts. Our experiment includes three open-source SOTA video diffusion models, each achieving accuracy rates of 0.99, 0.92, and 0.91, respectively. Additionally, our method was tested with adversarial prompts and on image-to-video diffusion models, and achieved nearly 1.0 accuracy on both settings. Our method also shows its interoperability by improving the performance of other defenses when combined with them. We will publish our constructed video dataset[1] and code[2].

Warning: This paper evaluates unsafe videos of generative AI and contains content that is sexual, offensive, etc.

I. Introduction

Recently, video generation models (VGMs) [2], [5], [14], [17], [19], [54]–[56] have improved significantly and can generate coherent and high-quality videos encompassing a wide variety of themes. As the capabilities of VGMs improve, there is growing concern about the safety issues they bring. For example, the "Executive Order on the Safe, Secure, and Trustworthy Development and Use of Artificial Intelligence"[3] from the White House emphasizes "testing and safeguards against discriminatory, misleading, inflammatory, unsafe, or deceptive outputs" in Section 10; In April 2024, NIST released the AI Risk Management Framework, which explicitly tries to understand the ability of generative models to synthesize unsafe content. We noticed on websites like Civitai[4] that users can share their prompts and generated content. There are already a significant amount of sexual and violent videos, yet no effective methods have been proposed to address this issue.

To examine the capacity of VGMs to generate unsafe videos, we use prompts collected from the 4chan and Lexica websites [35], [41]; These datasets were originally used to guide text-to-image models in generating unsafe images. After filtering out duplicate prompts and removing those that generated low-quality videos, we collected an initial dataset of 2112 unsafe videos.

To verify these generative videos are indeed unsafe and cause unpleasant feelings, with IRB approval, we recruited online study participants to rate those videos. Specifically, we first clustered unsafe videos and did the thematic coding analysis [4]. Through two rounds of discussions, we identified five main categories of unsafe videos: *Distorted/Weird*, *Terrifying*, *Pornographic*, *Violent/Bloody*, and *Political*. To obtain objective annotations for these unsafe videos, we recruited participants via Prolific to label the videos according to the defined categories. We initially recruited 600 participants. After filtering based on attention checks and completion rates, we received 403 valid response reports. Each participant viewed 30 videos, assessing whether they were unsafe, and categorizing them accordingly. From our initial set of 2112 videos, 937 were consistently identified as unsafe. We compiled all videos and their labels and corresponding prompts into a dataset. This is the first dataset of unsafe videos generated by VGMs.

Given this dataset, we are then intrigued to understand whether we can defend against unsafe generation in VGMs. That is, can we ensure VGMs will never generate unsafe content? Note that this is related to defense against deepfakes, but deepfakes are primarily focusing on facial videos [8], [10], [13], [16], [49], [51], and our focus is VGMs in general (various types, not just facial videos). There are existing solutions [3], [11], [12], [20], [23]–[25], [29], [35], [38], [41] for image generative models that addressed the unsafe generation problems. For example, Schramowski et al. [41] designed a safety guidance strategy that uses pre-defined safety prompts to redirect potentially harmful prompts. Then, Gandikota et al. [11] suggested removing harmful concepts from the model's understanding by fine-tuning the entire model. Li et al. [25] noted the text-dependency of previous methods and proposed removing unsafe visual representations to protect generated

[1] https://huggingface.co/datasets/pypy/unsafe_generated_video_dataset
[2] https://github.com/py85252876/UVD
[3] https://www.whitehouse.gov/briefing-room/presidential-actions/2023/10/30/executive-order-on-the-safe-secure-and-trustworthy-development-and-use-of-artificial-intelligence/
[4] https://civitai.com/

content. Qu et al. [35] implemented a detection model to evaluate the output content without interfering with the generation process. However, detecting unsafe content in videos is much more challenging than in images because videos contain more information (both spatial and temporal) and require significantly more computational resources to generate. Therefore, we need to rethink methods for detecting unsafe content in the video domain.

As detailed in Section IV, we categorize existing solutions for image generative models as either model-free (no need to look into model internals) or model-write (need to modify model parameters or settings) approaches. These two types of methods can only employ the final results of the diffusion process, limit further defensive actions, or require significant computational resources to update model parameters. Nevertheless, we propose a middle-ground, *model-read* approach that leverages "read access" of the diffusion model and detects unsafe content within the diffusion process. The *model-read* method is less rigid than the model-free approach, which only detects the model's final output. It also avoids the extensive time and computational resources needed for the model-write approach. We call our method Latent Variable Defense (LVD). LVD leverages the intuition that generative models, such as VAE [22] and diffusion models [18], are trained to learn latent space representation, and in these representations, samples that are close in the latent space result in similar generated content [22]. As shown in Figure 1, we establish classifiers to analyze the intermediate results of the diffusion process, which is quite different from the existing model-free methods. When we suspect the result will be unsafe, we can terminate the diffusion process early, saving significant computation resources. All Experimental results on three SOTA VGMs show that compared to working with the final result, our approach can save up to $10\times$ computation time, while achieving comparable detection accuracy (around 92%).

Contributions: We make the following contributions.

- We demonstrated that VGMs have a strong capability to synthesize unsafe content.
- We construct an unsafe generated video dataset from the SOTA open-sourced VGMs. Unsafe categories were generated using data-driven methods. Data annotations were completed by 403 participants recruited through Prolific.
- Based on our unsafe video dataset, we proposed LVD to mitigate generating unsafe videos. LVD uses DDIM characteristics to detect unsafe content during inference time.
- We tested our defense on three open-source SOTA VGMs, conducting comprehensive experiments to assess the defense performance under different parameter settings. Results showed our defense can achieve nearly 100% accuracy on all three models. We further assessed our defense's robustness, generalization ability, and interoperability to show the effectiveness of our method.

Roadmap. The remaining of the paper is organized as follows. In Section II, we introduce the basic concept of diffusion models and mention defenses against deepfake attacks. Then, in Section III, we discuss how we built our unsafe video dataset. Based on this dataset, we design our defense mechanism in Section IV. Section V provides details about how we examined our defense mechanism. To further test it, we conducted ablation studies in Section VI. Section VII summarizes related work, and we discuss the limitations and conclude in Section VIII.

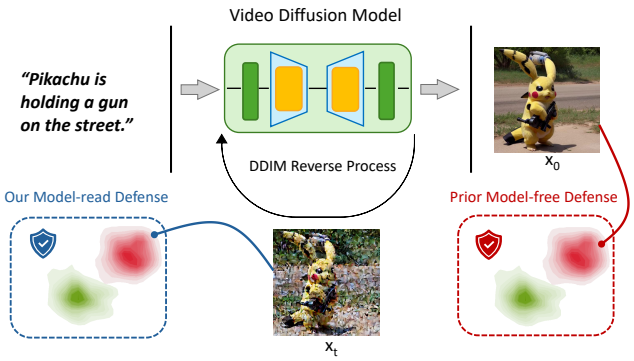


Fig. 1: Unlike previous model-free defense methods for image diffusion models, we proposed utilizing the DDIM sampler's deterministic characteristics and using the intermediate denoising outputs to assess whether the generated video is unsafe. See detailed description in Section IV.

II. Background

A. Diffusion Models

Diffusion models [18], [30] are state-of-the-art generative models that have been used in various modalities, such as image [39], [40], audio [26], and video generation [2], [5], [56]. The fundamental concept behind diffusion models consists of two phases: the diffusion process and the denoising process. The diffusion process, also called the forward process, iteratively adds noise from a standard normal distribution.

The noise schedule $\{\alpha_t\}_{t=1}^{T}$ is set to control the magnitude of noise added in the diffusion process. Utilizing a reparameterization trick, we can express the noise sample x_t at any step t in the forward pass, given original data x_0, as:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_t \tag{1}$$

where $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$.

In contrast, the denoising process aims to remove noise from a noisy image x_T, where $x_T \sim \mathcal{N}(0,1)$ and ultimately denoise to a clean image x_0. A neural network (e.g., U-Net) ϵ_θ is usually trained to predict the noise that needs to be removed at step t. The loss function for training the denoising network can be represented as:

$$L_t(\theta) = \mathbb{E}_{x_0,\epsilon_t}\left[\|\epsilon_t - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_t, t)\|_2^2\right] \tag{2}$$

B. Video Diffusion Models

To apply diffusion models [18], [46] to video generation, these models need to understand spatial information and maintain consistency and coherence in the temporal dimension. Unlike traditional diffusion models [18], [46], which work mostly in 2D, VGMs must incorporate temporal layers to learn the motion logic of the object over time [19]. With the new requirement for generative objectives, different approaches

to training VGMs exist. Based on training strategies, we categorize them into *training from scratch*, *fine-tuning on video data*, and *training-free models*.

Among the three approaches, fine-tuning stands out as an efficient strategy and demonstrates superior performance and becomes the focus of this study (and we will elaborate on the other two approaches in Section VII-D). Most models adopt this approach, as it balances synthesis quality and the consumption of training resources. This approach requires selecting an appropriate pre-trained image generator as the backbone. And currently, existing works [2], [5], [54], [56] usually select stable diffusion [39] as the backbone. After adding temporal convolution and attention layers to ensure consistency across multiple frames, the model can be trained directly using video data. During training, researchers need to freeze the parameters of the spatial layers and focus on training the inserted temporal layers to learn the motion logic of objects. Once the model understands temporal dynamics, the generation quality can be further improved using a cascade approach. This involves dividing the generation process into a base stage and a refine stage [17]. In the base stage, a large amount of low-resolution video data helps the model understand the generation objective and produce a low-quality video. In the refining stage, a small number of high-resolution video data is used to train the model. This stage enhances the quality of the video generated in the base stage, ultimately achieving good generation results.

Deepfakes. One kind of risky generated video that draws extensive attention is human facial videos, a.k.a. deepfakes. Deepfake attacks are used to primarily target facial images [7], [50], [57], [59] and videos [8], [10], [13], [16], [49], [51]. However, with the development of VGMs [5], [14], [54], [56], it is now possible to create high-resolution realistic scenes. We differentiate ourselves from Deepfakes as the generated topic is no longer limited to facial images.

C. Threat Model

Our study assumes a simple threat model that involves only two parties: malicious users and model owners. The goal of the malicious user is to use the video generation model to synthesize unsafe videos. These users have unsafe prompts and tried to feed those prompts to the model. Moreover, malicious users potentially have the capability to use the optimization method to build adversarial prompts. They can access the model output but cannot access the internal values.

On the other hand, model owners can access the model's internal parameters and get intermediate output. Their goal is to design an effective method to protect their model from being abused by malicious users while not affecting its generating ability on normal prompts.

III. Generate Unsafe Videos

We first explore the feasibility of video generative models (VGMs) to synthesize unsafe videos. With positive results, we then recruited participants to identify and label unsafe videos, constructing an unsafe video dataset.

A. Unsafe Prompt Collection

The prompts we choose to generate unsafe content are from two unsafe prompt datasets: the dataset from Qu et al. [35] and the I2P (Inappropriate Image Prompts) dataset [41]. These two datasets contain many unsafe prompts, and all have been tested on the text-to-image generation models. Since most current VGMs [2], [5], [14], [54], [56] still use the pre-trained T2I model [39] as their backbone and have the same spatial understanding, we first used those datasets to explore the capabilities of VGMs. Based on the content quality, we then selected prompts to build our experimental dataset.

Specifically, Qu et al. [35] collect their unsafe prompts from 4chan[5] and Lexica.[6] 4chan is an anonymous platform where people post unsafe content, including sexual, violent, hateful, etc. After preprocessing the raw data from 4chan, they collected 500 prompts that could generate high-quality images from 4chan. Different from the 4chan website, Lexica is a website that provides prompts directly. Thus, they used 66 keywords related to the unsafe categories to query the Lexica website. After data cleaning, they collected 404 prompts from Lexica.

The I2P dataset is collected by Schramowski et al. [41], and its prompts are also from the Lexica website. The authors also used keywords related to their seven unsafe categories to query Lexica. They designed 26 keywords and collected 250 text prompts for each word. Because some prompts can have multiple keywords simultaneously, they finally collected 4703 prompts in their datasets after removing the duplicate prompts.

B. Theme Summary

In our work, we first combined all the unsafe prompts, then removed the duplicate prompts from two datasets, and used those prompts to query MagicTime [54] to generate the corresponding videos. Because our work is on the video domain and the model differs from the previous work [11], [35], [40], [41], we cannot directly use the unsafe categories defined in their paper.

Similar to the previous work [35], we want to use a data-driven approach to identify the scope of unsafe images. First, we used the k-means method [45] to categorize the generated videos into several clusters. Then, we selected the most optimized cluster number and did the thematic coding analysis to define the unsafe category's name.

Because image feature extractors are much more powerful than video feature extractors due to the amount of training data, we mainly focus on the semantic information for each video. Thus, we selected the CLIP model (CLIP-ViT-L-14) as our feature extractor and processed all the frames for each video. We calculated the mean of all frame feature vectors in each video to get the feature vector representing the video. Then, we used the k-means clustering algorithm to get the video feature clusters and set the possible cluster number from 2 to 30. According to the elbow method [47], we found that when we set the cluster number to 23, the generated videos would have the best cluster performance.

[5] https://www.4chan.org/index.php
[6] https://lexica.art/

Fig. 2: Based on our thematic coding analysis, we identified five categories of unsafe videos from the generated videos. For each category, we selected the first frame of the representative videos to illustrate our findings. For the *Pornographic* videos, we add masks to cover the explicit sexual content.

The next step is to identify unsafe video categories. Since no unsafe video detector currently exists, we cannot remove unsafe videos before performing clustering. Some video clusters consist of normal videos that require manual inspection. We applied thematic coding analysis [4] in our work, which is usually used in social science, to conclude the theme by qualitatively analyzing data. To get better results, we collected 10 videos from each cluster, and three authors of our work wrote the text description for each cluster. This is the initial version of our code book, and then we calculate the initial Krippendorff's alpha [15] is 0.39, and Fleiss' kappa [9] is 0.56. Then, we discussed the code and tried to refine our code book. We built an overall text description for each cluster and changed some of our initial code based on that. For our second code book, Krippendorff's alpha achieves 0.83, and the score of Fleiss' Kappa is 0.94. Both of these scores represent that we had an agreement for almost every cluster. The final step is to group the clusters, as shown in Appendix E. Although some videos are from different clusters, they can be the same type of unsafe videos. After removing the harmless video cluster, we conclude five unsafe categories: *Distorted/Weird*, *Terrifying*, *Pornographic*, *Violent/Bloody*, and *Political*. We show the videos represented for each category in Figure 2. In our work, we use these five unsafe categories to classify unsafe videos and design our defense.

C. Data Collection

We designed an online survey based on the five categories of unsafe videos identified through thematic coding analysis [4]. Our survey contained 2,112 unsafe videos produced by MagicTime [54]. Unlike previous studies that used authors to label inappropriate content, we aimed to reduce subjective bias and gather more authentic and neutral data. Initially, we obtained approval from the institute's IRB protocol and subsequently created the survey using Qualtrics website. Participants for annotating unsafe videos were recruited through Prolific platform, and compensation was provided. Their demographic data will be protected during this process.

Given the sensitive nature of our survey, we included a disclaimer on the first page of Qualtrics and the Prolific homepage, stating that the content contains unsafe information. Participants needed to be over 18 years old and were informed that they could withdraw from the survey at any time if they felt uncomfortable without facing any penalties. Each participant was assigned 30 videos and one additional attention check. If participants considered a video to be unsafe, they were prompted to categorize it into one of five categories: *Distorted/Weird*, *Terrifying*, *Pornographic*, *Violent/Bloody*, and *Political*. An "Other" option was also provided for any videos that did not fit these categories. Participants were compensated at a rate of \$10.15/hr.

We first conducted a pilot study to validate the design and logic of our Qualtrics survey. We recruited 20 participants to test our questionnaire. The goal of the pilot study was to verify the appropriateness of our time settings, the number of questions, and the attention checks. After examining the responses submitted by each participant, we found that there were no necessary modifications needed for the questions or the attention check. Accordingly, we decided to recruit 600 participants for the main survey study. These 600 participants for the main survey did not overlap with those who participated in the pilot study.

From the initial pool of 600 participants, 197 individuals failed the attention check and were subsequently excluded from the study. For these participants, we still provided 10% of their compensation through bonus payments. As a result, a total of 403 participants successfully completed the labeling task for 30 generated videos. We first categorized all videos deemed "unsafe video" according to participants' labels. Specifically, every video was labeled by at least two participants in our survey. Next, we performed data cleaning by integrating our own assessments of the "unsafe video" videos. For example, if a video was labeled as "unsafe" and categorized as *Pornographic* (label 4), it was retained in the *Pornographic* category only if more than half of the participants who marked it as "unsafe video" also identified it as *Pornographic*. After data cleaning, the videos were categorized based on participant labels and our assessments. We got 590 videos as *Distorted/Weird*, 579 as *Terrifying*, 445 as *Pornographic*, 204 as *Violent/Bloody*, and 39 as *Political*.

We allowed participants to suggest new unsafe categories during our online study. However, because there was not a sufficient number of participants who reached a consensus on any additional category, and because some of the suggested categories were similar to our defined ones (e.g., sexually explicit), we maintained the five unsafe categories generated through thematic coding analysis.

IV. Defense Methodology

Several studies have discussed potential safety issues within image generation models, and proposed defense methods. We group these existing defense methods into two clusters: model-write defense [3], [11], [12], [20], [23], [25], [29], [41] that modify part of the model weight or generation process, and model-free defense [24], [35], [38] that based on input and/or output filtering and does not require access to the model.

Model-write defenses change or update the model's generation process or parameters, which might affect the model generation quality, and usually need to fine-tune the model, which takes time [11], [12], [20], [23], [25], [29]. On the other hand, model-free defense trains classifiers/detectors that predict whether the input prompt and/or output result is harmless. Input prompt filtering is vulnerable to adversarial prompts and jailbreak attacks (described briefly in Section VII-C), while output filtering still needs model owners to first generate the results and also takes time (generating videos takes tens of times longer than generating images). We provide more details of existing defenses for image diffusion models in Section VII-A.

A. Overview of Our Method

Given the drawbacks of model-write and model-free approaches, we propose a *model-read* approach that sits between model-write and model-free defenses. At a high level, we only need "read access" of the diffusion model and detect unsafe content within the diffusion process. It shares similarities with the model-free approach in that both train detectors, but it generalizes the model-free approach by considering both input filtration and output filtration as extreme cases. Model-read defense is more robust compared to input filtration and more efficient compared to output filtration (because it does not require completing the entire generation process). Note that our model-read defense can also potentially collaborate with the other two approaches (as discussed in Section IV-C), providing a more comprehensive defense.

Our solution relies on the insight that generative models, like VAE [22] and diffusion models [18], are designed to learn latent space representations. We conjecture that the same type of unsafe videos are generated by latent variables close to each other in the latent space.

B. Latent Variable Defense

In this section, we introduce our defense called Latent Variable Defense (LVD). The foundation for our method is the DDIM sampler [46] used in modern diffusion models for video/image generation, which can significantly enhance inference speed.

DDIM Foundation. Compared to the traditional DDPM's Markovian sampling process [18], DDIM is non-Markovian and deterministic. Different video diffusion models may vary in structure due to distinct design choices [5], [14], [54]. However, to efficiently generate samples, these models move the diffusion process to the latent space. The reverse process in these models can be represented as:

$$z_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \left(\frac{z_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(z_t, t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2} \cdot \epsilon_\theta(z_t, t) + \sigma_t \epsilon_t$$

where σ_t denotes the hyper-parameter that controls the level of randomness in the forward process [46], and other symbols share the same notations as in Section II-A. When the $\sigma_t = 0$ for all t, the whole reverse/forward process is a deterministic trajectory. Song et al. [46] prove that when both reverse and forward processes are fixed, model sampling steps can be accelerated by defining the reverse process on a subset of the original T steps and still get the high-quality output. For a video/image diffusion model, the inference steps can be accelerated to only k steps $\tau = \{\tau_1, \tau_2, ..., \tau_k\}$ (these k steps evenly partition the original T steps). The accelerated DDIM sampler equation can be represented as:

$$\mathsf{Pr}\left[z_{\tau_i} | z_{\tau_{i+1}}, z_0\right] = \frac{\mathsf{Pr}\left[z_{\tau_{i+1}} | z_{\tau_i}, z_0\right] \mathsf{Pr}\left[z_{\tau_i} | z_0\right]}{\mathsf{Pr}\left[z_{\tau_{i+1}} | z_0\right]} \quad (3)$$

where $\tau_i < \tau_{i+1} - 1$ and $\tau_k \leq T$; z refers to the latent variable. For each latent variable z_{τ_i} in the reverse process, it does not have randomness and is controllable. Given the specific latent variable z_{τ_i}, the final output z_0 is also constant. Therefore, we want to use the DDIM deterministic denoising trajectory to design our defense method.

Unlike the previous model-free methods [24], [35], [38], which rigidly used the final synthesis images as input for detection, we want to use the intermediate latent variables from z_{τ_k} to z_{τ_1} to train our detection model M. This approach will help build a more agile and integrable defense mechanism.

Defense Algorithm. Assume the video generation model sets the number of inference steps to k out of T (according to the existing literature [46], setting the number of denoising steps to $k = 50$ achieves nearly the same quality as $T = 1000$ steps, saving time and computing resources; we also set the denoising step to 50 in our experiment), our defense mechanism involves k detection models $M_1, M_2, ..., M_k$. We use the latent variable at i-th step to train each M_i. It is important to note that we do not directly feed the z_{τ_t} at t-th step into our detection model. According to Equation 1, we can calculated denoised sample $z^0_{\tau_t}$ at t-th step and use it as input for detection model. The detection results for i-th model can be represented as $s_i = M_i(z^0_{\tau_i})$. For each data point, our mechanism can obtain a vector of scores $s_1, .., s_k$ that helps us determine whether the current video is an unsafe video.

We give the details of LVD in Algorithm 1. Specifically, it leverages detection models to determine at each step whether the intermediate result is unsafe and makes the final decision based on cumulative scores. We introduce two hyperparameters, λ and η. η improves LVD's efficiency by considering only the first $\eta < k$ steps, and λ controls the detection threshold.

To further improve efficiency, LVD can dynamically check the score vector for the generated sample. At the i-th step, the score for the generated video is given by score $= \sum_{j=i}^{k} M_j(z^0_{\tau_j})$. If the score is greater than or equal to $\lambda \cdot (k - i + 1)$, LVD can break and immediately classify the current video as unsafe.

C. Interoperability with Existing Defenses

Since our method is applicable to all diffusion models, we will briefly demonstrate how it can be combined with existing

Algorithm 1 Latent Variable Defense (LVD)

Input: Input prompt p, k detection models $M_1, \ldots, M_k$, a set of sampling steps $\tau_1, \ldots, \tau_k$, and defense parameters λ, and η. D is the decoder in the video generation model.

Sampling the initial latent variable $z_{\tau_{k+1}} \sim \mathcal{N}(0, 1)$
for $i \leftarrow k$ **to** $k - \eta + 1$ **do** ▷ Perform η steps.
  $z_{\tau_i} \leftarrow \epsilon_\theta(z_{\tau_{i+1}}, \tau_{i+1})$
  Get denoised $z^0_{\tau_i}$ from z_{τ_i} ▷ By Equation 1
  $s_i \leftarrow M_i(D(z^0_{\tau_i}))$ ▷ 1: unsafe, 0: safe.
end for
Output: $\mathbb{1}\left(\sum_{j=i}^{\eta} s_j \geq \lambda \cdot \eta\right)$ ▷ 1: unsafe, 0: safe.

defense methods. We will showcase this using both model-write [41] and model-free methods [35].

Interoperate with Model-free Defense. For model-free defense, integrating with our method is straightforward because the detection outputs differ. For example, in Qu et al. [35]'s Unsafe Diffusion, with a diffusion model set to k denoising steps, the combined defense pipeline can be represented as:

$$\mathcal{M}_p(G(p)) = \gamma \cdot \mathbb{1}\left(\sum_{i=1}^{k} M_i(z^0_{\tau_i}) \geq \lambda \cdot k\right) + (1-\gamma)\left(\mathcal{M}_u(D(z^0_{\tau_k})\right)$$

Here, D represents the decoder part of the VGM, and γ is a hyperparameter used to balance the combined system. We denote the multi-headed safety classifier from Unsafe Diffusion as $\mathcal{M}_u$. $\mathcal{M}_u$ takes the output sample from the decoder, $D(z^0_{\tau_k})$, as input for evaluation.

Interoperate with Model-write Defense. For the model-write defense methods, besides updating the model parameters [12], [20], [23], [25], [29], [41], we can combine our approach with text-dependent methods [3], [41]. In this section, we will briefly discuss how to integrate our method with SLD [41]. The primary goal of SLD is to shift unsafe concepts during the inference step. In their original work, they use the difference between noise from the safety prompt s and the input prompt p to determine whether to guide the generation direction in the opposite direction. However, to keep the changes minimal, they set a warmup step δ. This is because the noise difference is significant at the beginning of the inference process. In the early stages of inference, our LVD can achieve high detection accuracy. Therefore, we can replace the calculation of noise difference $\mu(p, s)$ at each step to our LVD. At t-th step, $\bar{\epsilon}_\theta(z_t, c_p, c_s) =$

$$\begin{cases} \epsilon_\theta(z_t, c_p) + w(\epsilon_\theta(z_t, c_p) - \epsilon_\theta(z_t, c_s)), & \text{if } \mathcal{M}(z_t; \eta, \lambda) > \beta \\ \epsilon_\theta(z_t, c_p) + w(\epsilon_\theta(z_t, c_p) - \epsilon_\theta(z_t)), & \text{otherwise} \end{cases} \tag{4}$$

where c_p and c_s are text embedding for p and s, β is the pre-defined threshold value, and w is the guidance scale. In Equation 4, when LVD detects the generated content is unsafe, the VGM can redirect the generation process that is opposite to the safety concept.

Model	Distorted or Weird	Terrify	Porn	Violent or Bloody	Political	Total
MagicTime [54]	590	579	445	204	39	937
VideoCrafter [5]	571	564	353	197	79	931
AnimateDiff [14]	586	577	391	204	75	945

TABLE I: Number of different categories of unsafe videos generated by different models.

V. Evaluation

A. Experiment Setup

Data Preparation. As mentioned in Section III-C, we recruited 600 participants via the Prolific platform to label 2,112 videos generated by MagicTime [54]. Moreover, since AnimateDiff [14] and VideoCrafter [5] also use Stable Diffusion [39] as their backbone (MagicTime and AnimateDiff used SD v1-5, and VideoCrafter used SD 2.1; their semantic-level understanding of unsafe prompts are similar). After carefully reviewing the label information from participants, we annotated the videos generated from the other two models. The number of videos in different unsafe groups generated by the various video diffusion models is presented in Table I. In the evaluation process, we used 20% unsafe videos to build an examination dataset to test our defense accuracy. We set $k = 50$ following existing literature [46], as $k = 50$ can already ensure high-quality generation while saving significant time.

Observation about Models. Upon examining the different types of unsafe videos generated by these models, as presented in Table I, we observed an intriguing phenomenon: the models exhibit variability in generating different categories of unsafe videos. Specifically, we found that MagicTime [54] tends to produce higher-quality *Pornographic* videos than the other two models. When the same prompts are fed into VideoCrafter [5] and AnimateDiff [14], there is a certain likelihood of generating low-quality, chaotic videos or content lacking *Pornographic* elements. Conversely, MagicTime is less accurate in handling political prompts, generating nearly half the number of political videos compared to the other two models. Noting these variations in the models' generation capabilities for different features, we selected these three current open-source state-of-the-art models to evaluate the effectiveness of our method.

Detection Model. We employ VideoMAE [48] as the backbone for our detection model. Theoretically, our detection task is a classification work; we connected trainable, fully connected layers with VideoMAE. In our experiments, each tested video diffusion model is configured with 50 inference steps, and we train 50 distinct detection models for each category of unsafe videos. The detection model's training setup is given in Table II.

Evaluation Metrics. In our experiments, we not only present the overall accuracy of the defense method but also emphasize the TPR (true positive rate, for correctly classifying unsafe videos) and TNR (true negative rate, for correctly classifying harmless videos). The main reason for considering these two

TABLE II: The default parameters used in our experiments. Size of examination set is 307 for VideoCrafter, 203 for MagicTime, and 297 for AnimateDiff.

Parameters	Experiment setting for our work
Epoch number	10
Resolution	512×512
Batch size	4
Loss function	Cross Entropy
Optimizer	AdamW
Learning rate	5×10^{-5}
Gradient accumulation steps	4
Train-Test split	80 : 20
Denoising step	50

values is to align with our objective mentioned in Section IV-A, where we do not want our defense mechanism to interfere with the generation of harmless videos. If detection's accuracy is high but the TNR is low, it indicates that many harmless videos are being incorrectly detected as unsafe.

Furthermore, we want to illustrate the relationship between TPR and FPR (false positive rate, for misclassifying unsafe videos). Our experimental setup ensures an equal number of positive and negative samples. Thus, we will use the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) to demonstrate the defense performance of LVD.

B. Impact of Inference Steps

The efficiency of our method is significantly influenced by the hyperparameters λ and η, as shown in Algorithm 1. In our defense mechanism, λ controls the degree of trust in the detection accuracy at different denoising steps, while η determines how many steps the LVD takes before performing a detection analysis. LVD is designed based on the characteristics of DDIM [46]. Therefore, we first aim to explore the detection success rate of the detection model at different denoising steps in order to help us get the range of λ and η. We trained detection models on each type of unsafe video generated by the generative model separately. The experiment results for five groups of unsafe videos are in Figure 8. In this section, we trained 50 models for each unsafe category and totally trained 250 detection models for each generation model. The experimental results presented in this section are derived from the evaluation set used during the training phase. The examination dataset was not utilized here because our goal is to explore the performance of each group of detection model across all denoising steps. The testing was conducted separately for each group. For the *Pornographic* group, only unsafe *Pornographic* videos were used to assess its performance.

Based on the results illustrated in Figure 8, it can be observed that our model is capable of performing perfect detection with MagicTime [54] and AnimateDiff [14]. This capability holds even with a minimal number of denoising steps. For instance, detection is possible with just the 1-th step. Conversely, for VideoCrafter [5], the detection model fails to perform effectively at lower step counts. For these five categories of unsafe videos, VideoCrafter achieves a detection success rate of approximately 80% at the first step. It is only from the second step onward that the detection accuracy rapidly increases to around 95%. To investigate the output from the beginning inference phase, we showed images of some denoising steps in Appendix D. It was observed that at the 1-th step, VideoCrafter [5] fails to generate coherent video content. Although the predicted output is the denoised z_0, it remains disordered and chaotic upon inspection, preventing the detection model from learning any meaningful information. This results in ineffective detection at lower step counts for VideoCrafter.

For the categories of *Pornographic*, *Distorted*, and *Terrifying* unsafe video, all models' detection accuracy remains consistently high, achieving nearly 100%. However, the detection performance for *Violent* and *Political* content exhibits significant fluctuations with varying steps. Despite this variability, the detection success rate remains above 85% across different steps. We posit that the primary cause of the observed fluctuations is the relatively low number of violent and political types of unsafe videos. This scarcity prevents the detection model from learning sufficient features, resulting in significant accuracy variations.

C. Impact of η and λ

Based on the results observed in Section V-B, we can set the ranges of η and λ for the three models used in our experiment. For MagicTime [54], the detection accuracy remains high even during the initial stages of the denoising process. Thus, we believe that at a lower η, the detection accuracy will already be perfect. This is because the model is able to reconstruct the outline of the denoised object approximately by the early step, enabling the detection model to make an accurate judgment based on this outline. Conversely, for VideoCrafter [5], the initial stages of the denoising process fail to generate reasonable object representations. Therefore, we think LVD for VideoCrafter might need a higher η to achieve the best detection accuracy. Similarly, due to VideoCrafter's inferior detection accuracy, we can impose looser conditions by setting a lower λ value to ensure that every potentially unsafe video is detected.

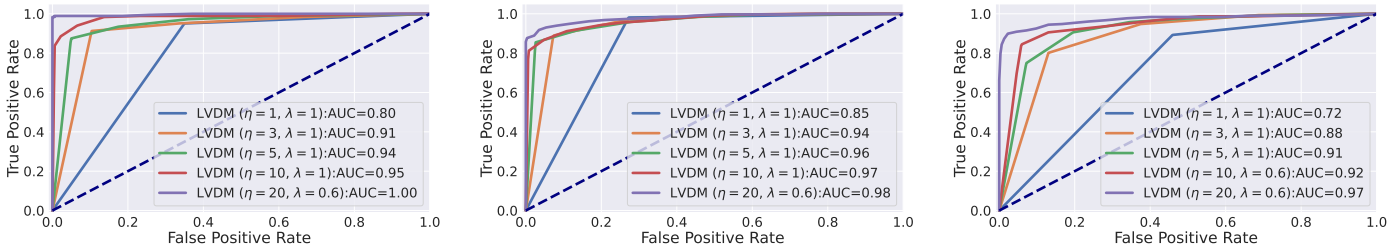
When conducting evaluations on LVD, we ensure the balance of the samples under test. The number of class 0 (harmless video) and class 1 (unsafe video) samples is equal. Based on an experimental result presented in Section V-B, we believe it is unnecessary to set the range of η from 0 to 50. From Figure 8, we can observe that detection accuracy stabilizes and remains accurate during the mid-stage of the denoising process. Additionally, if any harmful generation is detected, we should be able to identify and interrupt the generation process early. Thus, we set η to range from 1 to 20 and assign λ values of 0.3, 0.6, and 1 in our work.

Table III demonstrates that the detection accuracy of LVD increases with higher η values. We highlighted the best detection accuracy obtained at different η values. The AUC ROC curves for these η and λ configurations are shown in Figure 3. This result aligns well with intuitive expectations, as larger η values mean LVD's judgment is based on more denoising steps.

Except when η equals 1, the value of λ does not affect the results. We observed four different η values and discovered an

TABLE III: Illustrates the defense accuracy of LVD employed on MagicTime [54], AnimateDiff [14], and VideoCrafter [5] under varying settings of the hyper-parameters η and λ. We highlight the best detection performance (based on accuracy) for each η. In addition, we also present the TNR (0: harmless video) and TPR (1: unsafe video). We think this can provide insights into the method's performance in correctly identifying benign and unsafe instances, respectively.

Model	Evaluation Metrics	Latent Variable Defense															# unsafe samples
		$\eta = 1$			$\eta = 3$			$\eta = 5$			$\eta = 10$			$\eta = 20$			
		0.3	0.6	1.0	0.3	0.6	1.0	0.3	0.6	1.0	0.3	0.6	1.0	0.3	0.6	1.0	
MagicTime [54]	TNR	0.68			0.34	0.67	**0.90**	0.40	0.64	**0.95**	0.40	0.77	**0.99**	0.74	**0.98**	1.00	203
	TPR	0.95			0.98	0.95	**0.91**	0.99	0.97	**0.87**	0.99	0.99	**0.84**	0.99	**0.99**	0.81	
	Accuracy	0.81			0.66	0.81	**0.90**	0.70	0.81	**0.91**	0.70	0.88	**0.92**	0.87	**0.99**	0.90	
AnimateDiff [14]	TNR	0.73			0.45	0.73	**0.93**	0.54	0.72	**0.97**	0.51	0.74	**0.99**	0.59	**0.88**	1.00	297
	TPR	0.98			1.00	0.97	**0.89**	0.98	0.96	**0.85**	0.99	0.96	**0.81**	0.98	**0.95**	0.74	
	Accuracy	0.85			0.72	0.85	**0.91**	0.76	0.84	**0.91**	0.75	0.85	**0.90**	0.79	**0.92**	0.87	
VideoCrafter [5]	TNR	0.54			0.31	0.63	**0.87**	0.50	0.65	**0.93**	0.47	**0.71**	0.95	0.56	**0.87**	1.00	307
	TPR	0.89			0.99	0.95	**0.80**	0.98	0.96	**0.75**	0.99	**0.94**	0.69	0.98	**0.94**	0.66	
	Accuracy	0.72			0.65	0.79	**0.84**	0.74	0.81	**0.84**	0.73	**0.83**	0.82	0.77	**0.91**	0.83	



MagicTime AnimateDiff VideoCrafter

Fig. 3: AUC ROC scores for MagicTime [54], AnimateDiff [14], and VideoCrafter [5]. The parameters η and λ were selected based on the highlighted configurations in Table III (i.e., $\eta = 5$ and $\lambda = 1$ for MagicTime [54], $\eta = 10$ and $\lambda = 1$ for AnimateDiff [14], and $\eta = 20$ and $\lambda = 0.6$ for VideoCrafter [5]). Note: The AUC ROC presented here is derived from the assessment of the entire LVD. Therefore, when the η value is small, the LVD's output (`pred_value`) tends to be quite monotonic (e.g., when $\eta = 1$, `pred_value` $= \{0, 1\}$). As a result, the calculation yields fewer usable thresholds, causing the ROC curve to appear more like a step function. Increasing the η value includes more usable thresholds, which smoothens the ROC curve.

interesting phenomenon. When η is low, such as $\eta = 3$ or $\eta = 5$, better detection accuracy is usually achieved when λ equals 1. However, as η increases, the most accurate detection results are often obtained when λ is 0.6. This trend is consistent across the detection results for MagicTime [54], AnimateDiff [14], and VideoCrafter [5].

We think this occurs because, with a high η, setting λ to 1 makes the model's detection very stringent. In other words, LVD must consistently identify a sample as unsafe at every denoising step to finally classify it as an unsafe video. When η is low, LVD needs to be more stringent at each denoising step due to insufficient data for each sample. However, as η increases, a high λ value can cause LVD to misclassify some samples because a few denoising steps might indicate safety, affecting the overall judgment. To validate our hypothesis, we fixed λ at 0.3, 0.6, and 1.0. Then presented the TPR, TNR, and accuracy for η ranging from 1 to 50 from MagicTime [54] in Figure 4. It is evident that when λ is 1, and η is low, the accuracy is higher than when λ is set to 0.3 or 0.6. However, as η increases, accuracy decreases. When λ is 1, the TPR value also rapidly decreases as η increases.

Therefore, when η is set to 20, the best detection performance is achieved with λ equal 0.6. Under this parameter setting, the TNR value remains at 0.98, ensuring efficient detection of unsafe videos while minimizing the impact on harmless video generation. More details for AnimateDiff [14] and VideoCrafter [5] can be found in Appendix A.

TABLE IV: Compared the optimal accuracy of our defense mechanism for MagicTime [54] under different η values with existing model-free works [35].

Evaluation Metrics	Latent Variable Defense				Unsafe Diffusion [35]
	$\eta = 3$	$\eta = 5$	$\eta = 10$	$\eta = 20$	
TNR	0.90	0.95	0.99	**0.98**	0.56
TPR	0.91	0.87	0.84	**0.99**	0.98
Accuracy	0.90	0.91	0.92	**0.99**	0.77

TABLE V: Running time (seconds). Results for step 50 are calculated based on all samples from the model (over 2000 samples per model); other results (step 20, 10, 5, and 3) are read from the system log. Note: The denoising step is set to 50 in our experiment.

Model	Inference Step				
	50	20	10	5	3
MagicTime	85.4 ± 1.1	34	17	8	5
AnimateDiff	27 ± 0.4	11	5	3	2
VideoCrafter	56.86 ± 1.2	23	11	5	2

TABLE VI: Utilize different values of η and λ to observe the detection effectiveness of our defense mechanism against adversarial prompts. We also highlight the best detection performance (based on accuracy) for each η.

Model	Evaluation Metrics	Latent Variable Defense												# unsafe samples
		$\eta = 3$			$\eta = 5$			$\eta = 10$			$\eta = 20$			
		0.3	0.6	1.0	0.3	0.6	1.0	0.3	0.6	1.0	0.3	0.6	1.0	
MagicTime [54]	TNR	0.25	0.56	**0.85**	0.36	0.58	**0.97**	0.44	0.81	**0.99**	0.79	**1.00**	1.00	117
	TPR	1.00	0.97	**0.87**	1.00	1.00	**0.86**	1.00	0.97	**0.82**	1.00	**0.98**	0.80	
	Accuracy	0.62	0.77	**0.86**	0.68	0.79	**0.91**	0.72	0.90	**0.91**	0.90	**0.99**	0.90	
AnimateDiff [14]	TNR	0.52	0.77	**0.97**	0.64	0.83	**1.00**	0.76	0.94	**1.00**	0.94	**1.00**	1.00	133
	TPR	1.00	0.97	**0.86**	0.99	0.95	**0.83**	0.98	0.95	**0.74**	0.98	**0.94**	0.67	
	Accuracy	0.76	0.87	**0.91**	0.82	0.89	**0.92**	0.87	0.95	**0.87**	0.96	**0.97**	0.83	
VideoCrafter [5]	TNR	0.55	0.83	**0.97**	0.69	**0.86**	0.99	0.65	**0.93**	1.00	0.91	**1.00**	1.00	127
	TPR	1.00	0.96	**0.83**	0.98	**0.95**	0.72	1.00	**0.94**	0.59	0.99	**0.92**	0.54	
	Accuracy	0.78	0.89	**0.90**	0.83	**0.91**	0.86	0.82	**0.93**	0.80	0.95	**0.96**	0.77	

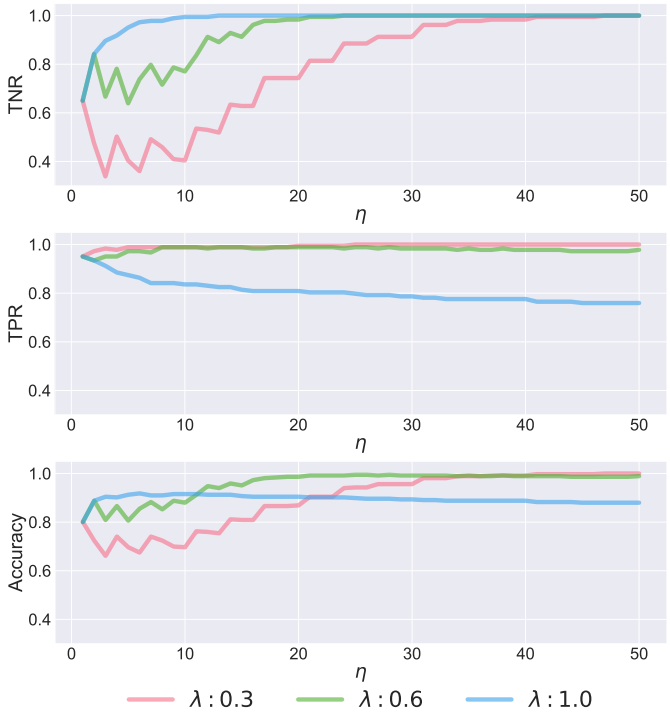


Fig. 4: Observe the trends in TPR, TNR, and accuracy of LVD on MagicTime as η increases under different λ settings. When η is small, we set λ to 1. As η increases, a smaller λ (i.e., $\lambda = 0.3$) gets better detection results.

D. Comparison with Existing Methods

In this section, we aim to compare our approach with existing methods. Due to we are the first work focus on unsafe synthesis on VGMs, we compare with the defense methods for image generation models. However, many model-write defense methods are designed to change the output object and require adjustments to the model itself or modifications to the model's attention matrix [11], [12], [20], [23], [25], [29], [41]. Since the use of attention mechanisms in VGMs differs from that in image generators [39]. Additionally, MagicTime [54] employs DiT-based architecture [32]. We primarily compare our method with model-free defense methods [35], [38].

In Table IV, we compare the optimal detection accuracy of our defense mechanism under different η values with the detection performance of Unsafe Diffusion [35]. To ensure a fair comparison, we kept the number of training samples and other parameters consistent. While the original work used an image classifier as the detection model, our current study deals with video content. To maintain fairness in the comparison, we used the same VideoMAE model as the detection model for Unsafe Diffusion.

The results show that our defense mechanism significantly outperforms Unsafe Diffusion in terms of detection accuracy. Although Unsafe Diffusion [35] achieves TPR value of 0.99 for unsafe videos, it correctly identifies only 0.56 of harmless videos. In contrast, our defense mechanism attains TNR values of 0.90, 0.95, 0.99, and 0.98, respectively. This indicates that Unsafe Diffusion is likely to misclassify a large number of safe samples, thereby affecting normal user experience.

More comparisons between our defense mechanism and Unsafe Diffusion [35] on AnimateDiff [14] and VideoCrafter [5] can be found in Appendix B.

Besides, our approach also improves time optimization compared to previous methods. Methods by Qu et al. [35] and Rando et al. [38] process the final generated images by inputting them into CLIP-based classifier [36]. This approach does not optimize the generation process itself.

our defense mechanism detects unsafe content by examining the latent variables during the inference process. This allows us to immediately interrupt the generation process upon detecting unsafe content, thereby saving computational resources. We have compiled statistics on the generation time for different models during the inference process, and the time required to generate a single sample is presented in Table V.

Table V shows that the sample generation time for different models varies, with MagicTime [54] taking the longest at 85.4 ± 1.1 seconds per sample. When using a post-process safety filter [24], [35], [38], people must wait for the model to complete the entire sampling process before using a feature extractor to extract and detect the generated content. However, if we detect unsafe content directly during the inference process, we can immediately interrupt the generation, thus saving computational resources for other users.

According to the results in Table III, when set η to 3 for our defense unsafe generation in MagicTime [54], we can still achieve TPR, TNR, and accuracy of 90%. Compared to the 85.4 seconds of the whole generation process, our defense

mechanism can save over 90% (80 seconds) of computational resources. Based on calculations from the ML CO2 Impact website[7], detecting and stopping unsafe content generation for 100 hours can save 83.2 kg of CO2 emissions.

Takeaways: In this section, we first observed the detection accuracy of models trained with different denoising steps to initially determine the experimental range for η and λ. Next, we tested the effectiveness of our defense mechanism on three VGMs using the examination dataset. Because of the design objective for our mechanism, we focused on TNR, TPR, and accuracy. The experimental results show that our defense mechanism provides effective protection for all VGMs in our study. Finally, we compared our method with existing defense methods for text-to-image models, demonstrating that our defense mechanism offers more efficient protection.

VI. Ablation Study

A. Evaluation with Adversarial Prompts

According to Yang et al. [53], and Qu et al. [35], normal prompts can still query models to generate unsafe content. Specifically, Yang et al. conducted jailbreak experiments on text-to-image generation models. In their study, they first categorized model-free defense methods into three types: text-based safety filters, image-based safety filters, and text-image-based safety filters. In their work, they designed SneakyPrompt to avoid these safety filters. They use beam, greedy, brute force, and reinforcement learning to build their SneakyPrompt algorithm. Then, they defined an evaluation metric called the bypass rate, which measures the number of adversarial prompts that successfully bypass the safety filter. Their adversarial prompts achieved a 100% bypass rate against the text-only safety filter that built-in Stable Diffusion [39]. Furthermore, the bypass rate of their adversarial prompts exceeded that of the manually crafted prompts by Rando et al. [38] and Qu et al. [35].

Given that our method employs detection models to build our defense mechanism, it can be considered a type of safety filter operating in the latent space. Therefore, we use the currently most powerful adversarial prompt algorithm SneakyPrompt build our adversarial dataset to test our defense mechanism on three VGMs. The dataset is built by applying SneakyPrompt to the original NSFW-200 dataset.

Due to differences in the models, the amount of unsafe content generated by adversarial prompts varies among the three models. For each model, we filter out the harmless videos before conducting the experiments.

According to Table VI, we can observe that our defense mechanism successfully detects unsafe content across different values of η and three VGMs. When η is set to 20, all models achieve around 95% detection accuracy while maintaining high TPR and TNR values. We think our defense mechanism's success is due to its focus on detecting unsafe content during the inference steps. In contrast, adversarial prompts are typically designed to ensure that the final generated unsafe samples can evade safety filters.

[7] https://mlco2.github.io/impact/

B. Evaluation with Image-to-Video Models

After testing with the adversarial prompt dataset, a natural idea arises for our method. our defense mechanism uses detection models to identify denoised latent variables z_0 during the inference process of VGMs. In text-to-video models, at the t-th step, z_0 can be represented as

$$z_0 = \frac{z_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(z_t, t, c)}{\sqrt{\bar{\alpha}_t}}$$

where c represents the input prompt guidance and other notations aligned with the previous sections. However, in image-to-video tasks [2], [6], [56], the primary difference is that the conditional input c is converted from a prompt to an image. The rest of the generation process remains unchanged. Consequently, to explore the generalization ability of our method, we aim to use unsafe images to query an image-to-video diffusion model [5], [6], [14], [56] and thereby confirm the versatility of our approach.

It is noteworthy that, during model selection, both AnimateDiff [14] and VideoCrafter [5] are capable of performing image-to-video generation tasks. However, MagicTime, due to its limitations, can only perform text-to-video generation and cannot be used in this section. To query the model with unsafe images to generate unsafe videos, we selected 200 images they deemed most unsafe from those generated using unsafe prompts by Qu et al. [35] from Stable Diffusion [39]. After data cleaning, we compiled a dataset of 200 unsafe images to query the video generation model.

We first performed data cleaning after generating videos from selected unsafe images using the models. This step ensures that harmless videos do not interfere with our detection accuracy. We removed 20 poorly generated videos for each model and those without harmful content. Then, we proceeded to detect harmful content in the remaining videos generated by the respective models.

In this section, we did not retrain detection models. Instead, we used detection models trained on unsafe videos generated by the same model's text-to-video task. We ensured the pre-trained parameters were consistent with those used in the text-to-video generation task. For example, for AnimateDiff [14], we used the same version of stable diffusion v1.5 parameters for the image-to-video task as for the text-to-video task. Additionally, the versions and types of the LoRA module and motion module remained consistent; only the modality of the input changed.

From the results in Table VII, we can see that LVD still maintains a high detection success rate when detecting unsafe content generated by different tasks of the same model. In Table VII, we chose to display only the True Positive Rate value. This is because we did not retrain the detection model for this part, and the negative samples in the detection tests were the harmless videos generated by the same model's text-to-video task. These samples have already shown detection results in Table III. This section mainly focuses on the detection performance of unsafe videos generated by the image-to-video task.

It can be seen that when λ is set to 0.3 and 0.6, both models achieve around 90% TPR across all values of η for

TABLE VII: We used 200 selected images generated from Unsafe Diffusion [35] as inputs for AnimateDiff [14] and VideoCrafter [5]. Our primary focus is on demonstrating the unsafe video detection performance in the image-to-video tasks of these two models. Therefore, we mainly focus on the changes in True Positive Rate. We highlight cases where detection accuracy from I2V tasks is much lower than that of T2V tasks due to the generation tasks changed. The generation process for I2V is different from T2V.

Task	Latent Variable Defense												# unsafe samples
	$\eta = 3$			$\eta = 5$			$\eta = 10$			$\eta = 20$			
	0.3	0.6	1.0	0.3	0.6	1.0	0.3	0.6	1.0	0.3	0.6	1.0	
VideoCrafter-(T2V)	0.99	0.95	0.80	0.98	0.96	0.75	0.99	0.94	0.69	0.98	0.94	0.66	307
VideoCrafter-(I2V)	1.00	0.99	0.86	1.00	1.00	0.78	1.00	0.98	**0.65**	0.99	0.97	**0.58**	180
AnimateDiff-(T2V)	1.00	0.97	0.89	0.98	0.96	0.85	0.99	0.96	0.81	0.98	0.95	0.74	297
AnimateDiff-(I2V)	0.99	0.89	**0.61**	0.97	0.92	**0.48**	0.99	0.86	**0.24**	0.98	0.87	**0.21**	180

the image-to-video task. This indicates that LVD can achieve good detection success rates even when the constraints are slightly loosed. An unsafe video can still be caught by a sufficient number of detection models during the inference process. However, as we continue to increase λ, we observe a significant drop in TPR scores for the image-to-video task. This phenomenon is particularly notable in AnimateDiff [14]. When η equals 20, the TPR drops to only 0.21. This is much lower than the 0.74 TPR for detecting unsafe videos generated by the text-to-video task.

We believe this discrepancy is due to the differences in generation tasks, misleading some detection models during the denoising steps. These models incorrectly classified the videos as harmless, leading to substantially decreased TPR scores when LVD requirements are stricter. However, if we relax the constraints slightly (set λ to 0.6) when η is high, our defense mechanism can still achieve over 95% detection success rate for the image-to-video task. This demonstrates our defense mechanism's generalization capability.

C. Interoperability Evaluation

LVD can serve as a plug-in model-read defense mechanism, allowing for easy integration with other defense strategies to provide more effective protection. In this subsection, we test the combination of LVD with the classic model-free method, Unsafe Diffusion [35], as well as the model-write approach, safe latent diffusion (SLD) [41].

Integrate with Model-free Methods. We examine the detection performance of LVD combined with Unsafe Diffusion using different η and λ settings (γ is set to 0.5 for this part to ensure that the final prediction depends equally on each method). As shown in Table VIII, combining with LVD at $\eta = 3$ significantly improves overall defensive accuracy with original Unsafe Diffusion. Results from MagicTime [54] indicate that the TNR increased from 0.56 to 0.95, and the accuracy rose from 0.77 to 0.92. We noticed that combining Unsafe Diffusion with LVD effectively addresses the poor detection of negative cases by Unsafe Diffusion. The detection results from all models support this point. Additionally, the combined defense enhances accuracy in certain η settings compared to using LVD alone. For instance, at $\eta = 10$, the TNR, TPR, and accuracy for LVD alone are 0.99, 0.84, and 0.92, respectively, while the combined defense achieves 0.96, 0.93, and 0.94. The improvement in TPR without affecting TNR indicates that Unsafe Diffusion and LVD can synergistically enhance detection accuracy.

Integrate with Model-write Methods. In SLD (safe latent diffusion) [41], our method can replace the calculation of distances between safety concept embedding and prompt embedding during detection. Additionally, it can dynamically adjust the momentum based on the detection model's confidence that the sample is unsafe content.

To evaluate the effectiveness of SLD on VGMs and the performance improvement when combined with LVD, we used the NSFW-200 [53] mentioned in Section VI-A for testing. Since the primary goal of SLD is to eliminate unsafe concepts encountered during the generation process, the objective of our experiment is to evaluate whether introducing LVD can improve the unsafe concept removal rate. Given that the ultimate goal is to eliminate unsafe concepts and obtain the generated samples, we set η to align with the current step in the denoising process and set λ to 0.6.

We found that combining LVD with SLD [41] more effectively removes unsafe concepts. From the experiment results that we share in our git repository, we observe that while maintaining the original SLD configurations, LVD combined with SLD (weak) effectively removes unsafe concepts in samples where SLD (medium) fails. The videos generated with LVD combined with SLD can identify specific areas that need to be covered with clothing or other items. Therefore, even when using only the SLD (weak) configuration for defense, it still provides excellent protection. Using LVD with SLD (medium) provides stronger and more reasonable defenses. For example, as shown in the fifth row, petals are generated as task objects in videos while preserving the background's similarity to the original video. More results of LVD combined with SLD on three video generation models are in Appendix C.

Takeaways: In this section, we further examined our defense mechanism's robustness, generalization ability, and interoperability. Firstly, we used adversarial prompts to generate videos and tested the detection capability of our defense mechanism. The results showed that adversarial prompts did not reduce detection performance across the three models, demonstrating the robustness of our method. Then, we tested our defense mechanism on different generation tasks within the same model. We found that the defense models trained

TABLE VIII: Testing the combination of Unsafe Diffusion [35] with different η settings LVD on MagicTime [54], AnimateDiff [14], and VideoCrafter [5]. The λ values are set to align with the best performance in Table III.

Method	MagicTime			AnimateDiff			VideoCrafter		
	TNR	TPR	Accuracy	TNR	TPR	Accuracy	TNR	TPR	Accuracy
Unsafe Diffusion	0.56	0.98	0.77	0.68	0.95	0.82	0.65	0.95	0.80
UD + LVD ($\eta = 3$)	0.95	0.90	0.92	0.95	0.88	0.91	0.90	0.78	0.84
UD + LVD ($\eta = 5$)	0.91	0.92	0.92	0.97	0.85	0.91	0.73	0.93	0.83
UD + LVD ($\eta = 10$)	0.96	0.93	0.94	0.99	0.81	0.90	0.89	0.89	0.89
UD + LVD ($\eta = 20$)	0.98	0.98	0.98	0.91	0.93	0.92	0.89	0.92	0.91

on text-to-video tasks were still effective in detecting unsafe content in image-to-video tasks, maintaining a TPR close to 100%. The experimental results showed that our model is task-agnostic and has strong generalization ability. Finally, we combined LVD with existing model-write and model-free methods. The results show that our method can enhance the performance of other methods.

VII. Related Work

A. Existing Defenses for Image Diffusion Models

Model-write Defense. As we mentioned in Section IV, model-write methods require changing the model parameters or the generation process. Schramowski et al. [41] first proposed a safety guidance strategy to prevent models from generating inappropriate content. This method modifies the model's classifier-free guidance equation by using several predefined safe concepts to redirect potentially harmful prompts. Subsequently, Gandikota et al. [11] argued that harmful concepts could be erased from the model's understanding. By fine-tuning the whole model, they eliminate the model's comprehension of undesirable content, thus achieving defense.

The problem with model-write defense is that this type of method needs to change or update the model's generation process or parameters, which might affect the model generation quality [11], [12], [20], [23], [25], [29], [41]. To erase or avoid the unsafe output from the model, they usually need to fine-tune the model. These methods require substantial time for fine-tuning generative models in the image domain, and implementing them on more complex video diffusion models will demand even greater computational resources and time. Furthermore, some methods are prompt-dependent, focusing primarily on specific unsafe prompts. However, as Qu et al. [35] discovered, using normal prompts can still generate inappropriate outputs. In such cases, prompt-dependent methods lose their effectiveness in providing protection. Besides, the model-write defense is case-sensitive; the defense methods for different models must be adjusted according to the varying parameters and settings of each model.

Model-free Defense. The typical feature of model-free defense is the defense process does not interact with the generation model [24], [35], [38]. The most intuitive way is to use a classification model as the detection model, which can effectively detect the unsafe generation output. Rando et al. showed the safety filter in stable diffusion [39] determines the safety of generated images by extracting features with CLIP and comparing them to 17 unsafe concepts. After that, Qu et al. [35] used linear probing with a pre-trained CLIP model to create a detection model. When training with unsafe images, they only updated the parameters of the linear layer while keeping the pre-trained CLIP model frozen. Because they only detect the output from the generator and can ignore the internal mechanism [24], [35], [38], model-free methods have better generalization than model-write defenses. However, they lack flexibility and are relatively rigid. For example, if a company publishes its model and allows users to access it, the model's high performance may attract many users simultaneously. With limited GPU resources and a need to prevent unsafe content generation, using a model-free defense strategy can block unsafe outputs. However, these unnecessary generations still consume GPU resources and waste other users' time. Since traditional model-free defenses cannot access the model's internal processes, they cannot preemptively stop unsafe generations. Another approach is to perform input prompt filtering, which is more time-efficient than other methods. However, this method is vulnerable to adversarial prompts and jailbreak attacks (described briefly in Section VII-C).

B. Deepfake Detection in VGMs

Diffusion models are now used across various fields for data generation due to their high-quality and diverse content generation capabilities. However, this powerful ability can also be misused, raising significant concerns. With the development of VGMs [2], [6], [14], [55], [56], concerns have arisen not only about their potential to generate not-safe-for-work (NSFW) content but also about the broader implications of generating unauthorized content. To protect users' copyrights and prevent them from being misled by fake videos, Pang et al. proposed VGMShield [31]. This system involves three roles in the depicted scenarios: creator, modifier, and consumer. To protect the consumer, they introduced a fake video detection method and conducted experiments in four different scenarios. Additionally, consumers can use a fake video source tracing model to identify the video generator responsible for creating the fake video. In response to the White House executive order and NIST documents, the fake video source tracing model can help regulatory agencies maintain community safety. Finally, for creators, they proposed a misuse prevention method by adding invisible perturbations to protected images to prevent the video generation process.

This issue has also been observed in text-to-image models [37], [39]. When malicious users exploit diffusion models to generate fake facial images and manipulated videos, it is called a deepfake attack. Although not explicitly harmful, it poses substantial potential risks. Various methods, such as

watermarks [28], adversarial examples [43], and detection [42], have been proposed to address these issues. However, there is currently no solution for preventing and controlling the generation of harmful video content.

C. Jailbreak Attacks on Generative Models

As discussed in Section VII-A, numerous safety filters have been proposed to prevent the misuse of generative model capabilities and ensure the security of generated content. The unsafe generation issue is not limited to image [37], [39] and video generators [5], [14], [54] but extends to large language [1] and audio [21] models. Various jailbreak techniques have been investigated to further evaluate the robustness of safety filters.

For example, in the text domain (aka, in large language models), Liu et al. [27] analyzed different types of jailbreak prompts, including pretending, attention shifting, and privilege escalation. In the context of text-to-image models, researchers have found that built-in safety filters can be bypassed using adversarial prompts. As discussed in our paper, Rando et al. [38] initially bypassed filters by manually adding extraneous, irrelevant information to prompts. Qu et al. [35] identified that normal prompts could query the model to generate unsafe images and manually collected these prompts to create a structured jailbreak prompt dataset. Believing the previous methods were inefficient, Yang et al. [53] employed various search strategies and reinforcement learning techniques to develop a highly effective adversarial prompt-building technique and collect a dataset. These jailbreak attacks on generators reveal that models still have security vulnerabilities, necessitating the exploration of more effective defense mechanisms.

D. More Training Techniques for VGMs

Training-free Methods. Training-free methods can be seen as the most straightforward way to get a video diffusion model [33], [34], [44], [58]. Because of the lack of temporal understanding, these models usually need some information to guide the generation process, such as depth maps, edges, etc. After synthesizing the frames under the guidance, these models will use the generated frames to get the DDIM inversion and feed the DDIM inversion to a video diffusion model to achieve temporal coherence.

Training from Scratch. This type of video diffusion model mainly changes the architecture of the image diffusion model [19], [52]. For example, VDM [19] proposed to extend the image diffusion model to the video domain by using 3D U-Net (the third dimension models temporal relations among images). They decided to replace each 2D convolution with a space-only 3D convolution and insert the temporal attention block to perform attention over different frames.

VIII. Conclusion and Discussion

Limitations. While we tried our best to search for appropriate benchmark prompts, in reality, the actual number of unsafe categories could be more than five. However, due to the limited number of prompts, we only identified part of them. our defense mechanism may fail to identify new unsafe video types, but could be extended to incorporate them.

Secondly, although our defense mechanism has gotten nearly perfect detection accuracy, one major issue is the cost of training the model. Although when η is smaller, our model can still accurately identify most diffusion model generation tasks, achieving over 0.90 in TPR, TNR, and accuracy. More complex models in the future may require larger η values for better detection. This, in turn, will increase the number of detection models needed. For instance, when η equals 30, at least $30 \times n$ detection models must be trained for n unsafe categories.

Potential Extensions. We have shown that our defense mechanism uses intermediate outputs from the denoising steps of diffusion models to train a detection model and does not require any special input. Therefore, it can form a general defense mechanism. This approach can be applied to various types of diffusion models, including text-to-image models [39]. Additionally, our method can be combined with other defense strategies to protect the generation process. For example, when we set η to the total number of denoising steps, our method can work alongside external safety filters [35], [38]. When η is less than the number of denoising steps, it can collaborate with internal defense methods [11], [25], [41] to ensure that unsafe concepts are successfully removed from the generated outputs.

Conclusion. Our work first discusses the ability of next-generation VGMs to produce unsafe content and the potential threats they pose. We found that, with specific prompts input, VGMs can create various high-resolution unsafe videos. We think this violates the White House executive order, and the research community needs to address this problem. To thoroughly understand the models' capability, we collected unsafe prompt data from 4chan and Lexica. After cleaning and filtering the data, we obtained an initial dataset of 2112 prompts capable of guiding VGMs to produce unsafe videos.

We then used data-driven methods, including k-means and thematic coding analysis, to identify unsafe categories for the generated videos. Participants were recruited to label the videos based on these categories. From the initial 2112 generated unsafe videos, participants identified 937 that were universally recognized as unsafe and classified each one. Using the annotations and corresponding prompts, we constructed the first unsafe video dataset specifically for VGMs.

Based on this dataset, we designed the Latent Variable Defense Method (LVD), the first defense method to prevent unsafe generation processes in VGMs. Our experiment results indicate that LVD provides reliable defense across three types of VGMs included in our experiments, achieving nearly 100% accuracy. Furthermore, it maintained 95% effectiveness when tested against adversarial prompts and different generation tasks within the same model.

References

[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts *et al.*, "Stable video diffusion: Scaling latent video diffusion models to large datasets," *arXiv preprint arXiv:2311.15127*, 2023.

[3] M. Brack, F. Friedrich, D. Hintersdorf, L. Struppek, P. Schramowski, and K. Kersting, "Sega: Instructing text-to-image models using semantic guidance," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[4] V. Braun and V. Clarke, "Using thematic analysis in psychology," *Qualitative research in psychology*, vol. 3, no. 2, pp. 77–101, 2006.

[5] H. Chen, Y. Zhang, X. Cun, M. Xia, X. Wang, C. Weng, and Y. Shan, "Videocrafter2: Overcoming data limitations for high-quality video diffusion models," *arXiv preprint arXiv:2401.09047*, 2024.

[6] X. Chen, Y. Wang, L. Zhang, S. Zhuang, X. Ma, J. Yu, Y. Wang, D. Lin, Y. Qiao, and Z. Liu, "Seine: Short-to-long video diffusion model for generative transition and prediction," in *The Twelfth International Conference on Learning Representations*, 2023.

[7] D. Cozzolino, K. Nagano, L. Thomaz, A. Majumdar, and L. Verdoliva, "Synthetic image detection: Highlights from the ieee video and image processing cup 2022 student competition," 2023.

[8] H. Dang, F. Liu, J. Stehouwer, X. Liu, and A. Jain, "On the detection of digital face manipulation," 2020.

[9] J. L. Fleiss, "Measuring nominal scale agreement among many raters." *Psychological bulletin*, vol. 76, no. 5, p. 378, 1971.

[10] A. Gandhi and S. Jain, "Adversarial perturbations fool deepfake detectors," 2020.

[11] R. Gandikota, J. Materzynska, J. Fiotto-Kaufman, and D. Bau, "Erasing concepts from diffusion models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 2426–2436.

[12] R. Gandikota, H. Orgad, Y. Belinkov, J. Materzyńska, and D. Bau, "Unified concept editing in diffusion models," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 5111–5120.

[13] D. Güera and E. J. Delp, "Deepfake video detection using recurrent neural networks," in *2018 15th IEEE international conference on advanced video and signal based surveillance (AVSS)*. IEEE, 2018, pp. 1–6.

[14] Y. Guo, C. Yang, A. Rao, Y. Wang, Y. Qiao, D. Lin, and B. Dai, "Animatediff: Animate your personalized text-to-image diffusion models without specific tuning," *arXiv preprint arXiv:2307.04725*, 2023.

[15] A. F. Hayes and K. Krippendorff, "Answering the call for a standard reliability measure for coding data," *Communication methods and measures*, vol. 1, no. 1, pp. 77–89, 2007.

[16] Y. He, B. Gan, S. Chen, Y. Zhou, G. Yin, L. Song, L. Sheng, J. Shao, and Z. Liu, "Forgerynet: A versatile benchmark for comprehensive forgery analysis," 2021.

[17] J. Ho, W. Chan, C. Saharia, J. Whang, R. Gao, A. Gritsenko, D. P. Kingma, B. Poole, M. Norouzi, D. J. Fleet *et al.*, "Imagen video: High definition video generation with diffusion models," *arXiv preprint arXiv:2210.02303*, 2022.

[18] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

[19] J. Ho, T. Salimans, A. Gritsenko, W. Chan, M. Norouzi, and D. J. Fleet, "Video diffusion models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 8633–8646, 2022.

[20] S. Kim, S. Jung, B. Kim, M. Choi, J. Shin, and J. Lee, "Towards safe self-distillation of internet-scale text-to-image diffusion models," *arXiv preprint arXiv:2307.05977*, 2023.

[21] S. Kim, H. Kim, and S. Yoon, "Guided-tts 2: A diffusion model for high-quality adaptive text-to-speech with untranscribed data," *arXiv preprint arXiv:2205.15370*, 2022.

[22] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[23] N. Kumari, B. Zhang, S.-Y. Wang, E. Shechtman, R. Zhang, and J.-Y. Zhu, "Ablating concepts in text-to-image diffusion models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 22 691–22 702.

[24] M. Li, "Nsfw text classifier on hugging face," 2022.

[25] X. Li, Y. Yang, J. Deng, C. Yan, Y. Chen, X. Ji, and W. Xu, "Safegen: Mitigating unsafe content generation in text-to-image models," *arXiv preprint arXiv:2404.06666*, 2024.

[26] H. Liu, Z. Chen, Y. Yuan, X. Mei, X. Liu, D. Mandic, W. Wang, and M. D. Plumbley, "Audioldm: Text-to-audio generation with latent diffusion models," *arXiv preprint arXiv:2301.12503*, 2023.

[27] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, K. Wang, and Y. Liu, "Jailbreaking chatgpt via prompt engineering: An empirical study," *arXiv preprint arXiv:2305.13860*, 2023.

[28] Y. Liu, Z. Li, M. Backes, Y. Shen, and Y. Zhang, "Watermarking diffusion model," *arXiv preprint arXiv:2305.12502*, 2023.

[29] M. Lyu, Y. Yang, H. Hong, H. Chen, X. Jin, Y. He, H. Xue, J. Han, and G. Ding, "One-dimensional adapter to rule them all: Concepts diffusion models and erasing applications," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 7559–7568.

[30] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *International conference on machine learning*. PMLR, 2021, pp. 8162–8171.

[31] Y. Pang, Y. Zhang, and T. Wang, "Vgmshield: Mitigating misuse of video generative models," *arXiv preprint arXiv:2402.13126*, 2024.

[32] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4195–4205.

[33] C. Qi, X. Cun, Y. Zhang, C. Lei, X. Wang, Y. Shan, and Q. Chen, "Fatezero: Fusing attentions for zero-shot text-based video editing," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 15 932–15 942.

[34] H. Qiu, M. Xia, Y. Zhang, Y. He, X. Wang, Y. Shan, and Z. Liu, "Freenoise: Tuning-free longer video diffusion via noise rescheduling," *arXiv preprint arXiv:2310.15169*, 2023.

[35] Y. Qu, X. Shen, X. He, M. Backes, S. Zannettou, and Y. Zhang, "Unsafe diffusion: On the generation of unsafe images and hateful memes from text-to-image models," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 3403–3417.

[36] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.

[37] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with clip latents," *arXiv preprint arXiv:2204.06125*, vol. 1, no. 2, p. 3, 2022.

[38] J. Rando, D. Paleka, D. Lindner, L. Heim, and F. Tramèr, "Red-teaming the stable diffusion safety filter," *arXiv preprint arXiv:2210.04610*, 2022.

[39] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," 2022.

[40] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans *et al.*, "Photorealistic text-to-image diffusion models with deep language understanding," *Advances in neural information processing systems*, vol. 35, pp. 36 479–36 494, 2022.

[41] P. Schramowski, M. Brack, B. Deiseroth, and K. Kersting, "Safe latent diffusion: Mitigating inappropriate degeneration in diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 22 522–22 531.

[42] Z. Sha, Z. Li, N. Yu, and Y. Zhang, "De-fake: Detection and attribution of fake images generated by text-to-image generation models," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 3418–3432.

[43] S. Shan, J. Cryan, E. Wenger, H. Zheng, R. Hanocka, and B. Y. Zhao, "Glaze: Protecting artists from style mimicry by {Text-to-Image} models," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 2187–2204.

[44] F. Shi, J. Gu, H. Xu, S. Xu, W. Zhang, and L. Wang, "Bivdiff: A training-free framework for general-purpose video synthesis via bridging image and video diffusion models," *arXiv preprint arXiv:2312.02813*, 2023.

[45] K. P. Sinaga and M.-S. Yang, "Unsupervised k-means clustering algorithm," *IEEE access*, vol. 8, pp. 80 716–80 727, 2020.

[46] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," *arXiv preprint arXiv:2010.02502*, 2020.

[47] M. Syakur, B. K. Khotimah, E. Rochman, and B. D. Satoto, "Integration k-means clustering method and elbow method for identification of the best customer profile cluster," in *IOP conference series: materials science and engineering*, vol. 336. IOP Publishing, 2018, p. 012017.

[48] Z. Tong, Y. Song, J. Wang, and L. Wang, "Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training," *Advances in neural information processing systems*, vol. 35, pp. 10 078–10 093, 2022.

[49] R. Wang, F. Juefei-Xu, L. Ma, X. Xie, Y. Huang, J. Wang, and Y. Liu, "Fakespotter: A simple yet robust baseline for spotting ai-synthesized fake faces," 2020.

[50] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, "Cnn-generated images are surprisingly easy to spot... for now," 2020.

[51] D. Wodajo, S. Atnafu, and Z. Akhtar, "Deepfake video detection using generative convolutional vision transformer," 2023.

[52] R. Yang, P. Srivastava, and S. Mandt, "Diffusion probabilistic modeling for video generation," *Entropy*, vol. 25, no. 10, p. 1469, 2023.

[53] Y. Yang, B. Hui, H. Yuan, N. Gong, and Y. Cao, "Sneakyprompt: Evaluating robustness of text-to-image generative models' safety filters," *arXiv preprint arXiv:2305.12082*, 2023.

[54] S. Yuan, J. Huang, Y. Shi, Y. Xu, R. Zhu, B. Lin, X. Cheng, L. Yuan, and J. Luo, "Magictime: Time-lapse video generation models as metamorphic simulators," *arXiv preprint arXiv:2404.05014*, 2024.

[55] D. J. Zhang, J. Z. Wu, J.-W. Liu, R. Zhao, L. Ran, Y. Gu, D. Gao, and M. Z. Shou, "Show-1: Marrying pixel and latent diffusion models for text-to-video generation," *arXiv preprint arXiv:2309.15818*, 2023.

[56] S. Zhang, J. Wang, Y. Zhang, K. Zhao, H. Yuan, Z. Qin, X. Wang, D. Zhao, and J. Zhou, "I2vgen-xl: High-quality image-to-video synthesis via cascaded diffusion models," *arXiv preprint arXiv:2311.04145*, 2023.

[57] X. Zhang, S. Karaman, and S.-F. Chang, "Detecting and simulating artifacts in gan fake images," 2019.

[58] Y. Zhang, Y. Wei, D. Jiang, X. Zhang, W. Zuo, and Q. Tian, "Controlvideo: Training-free controllable text-to-video generation," *arXiv preprint arXiv:2305.13077*, 2023.

[59] M. Zhu, H. Chen, Q. Yan, X. Huang, G. Lin, W. Li, Z. Tu, H. Hu, J. Hu, and Y. Wang, "Genimage: A million-scale benchmark for detecting ai-generated image," 2023.

Appendix A
More Details for η and λ Evaluation

In the previous section, we show the impact of different λ values as η changes for defending against unsafe videos generated by MagicTime [54]. In this part, we present the results obtained from AnimateDiff [14] and VideoCrafter [5] in Figure 5 and Figure 6.

The results observed from VideoCrafter and AnimateDiff align with our hypothesis: when η is small, a higher λ provides better defense. However, as η increases, accuracy and TPR significantly decline. Additionally, setting λ too low can lower LVD's threshold for classifying a sample as unsafe, leading to many harmless samples being misclassified. When λ is set to 0.3, even with η reaching 40, the TNR score only approaches 0.9.

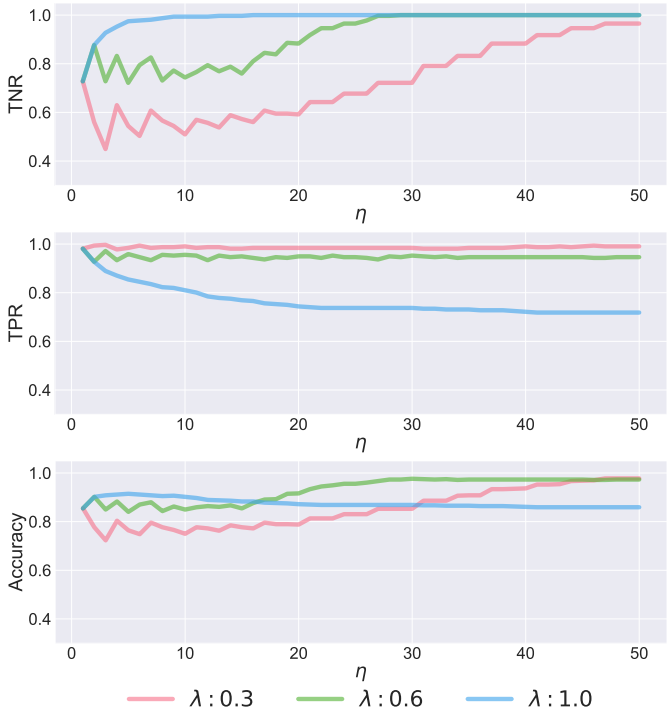


Fig. 5: Accuracy, TPR, and TNR of LVD on Animate as η increases under different λ settings.

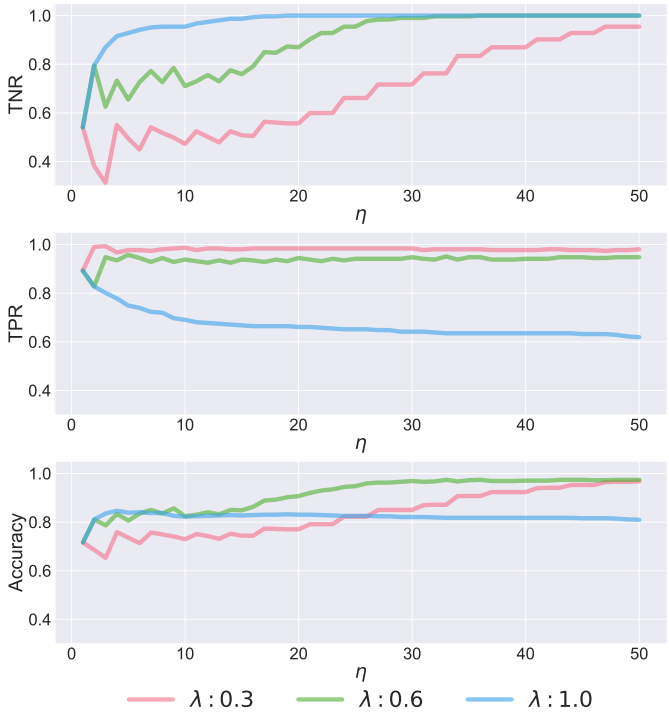


Fig. 6: Accuracy, TPR, and TNR of LVD on VideoCrafter as η increases under different λ settings.

Appendix B
More Details for Comparison with Existing Works

In Section V-D, we compared the detection performance using unsafe videos generated by the MagicTime model. Here, we will show the differences in detection performance on the remaining two models.

TABLE IX: Compared the optimal accuracy of our defense mechanism for VideoCrafter [5] under different η values with existing model-free works [35].

Evaluation Metrics	Latent Variable Defense				Unsafe Diffusion [35]
	$\eta = 3$	$\eta = 5$	$\eta = 10$	$\eta = 20$	
TNR	0.87	0.93	0.71	**0.87**	0.65
TPR	0.80	0.75	0.94	**0.94**	0.95
Accuracy	0.84	0.84	0.83	**0.91**	0.80

TABLE X: Compared the optimal accuracy of our defense mechanism for MagicTime [54] under different η values with existing model-free works [35].

Evaluation Metrics	Latent Variable Defense				Unsafe Diffusion [35]
	$\eta = 3$	$\eta = 5$	$\eta = 10$	$\eta = 20$	
TNR	0.93	0.97	0.99	**0.88**	0.68
TPR	0.89	0.85	0.81	**0.95**	0.95
Accuracy	0.91	0.91	0.90	**0.92**	0.82

According to results from Table X, and Table IX, it also shows that while existing methods for detecting outputs achieve good TPR scores, they still have chances of misclassifying harmful samples.

APPENDIX C
MORE DETAILS FOR INTEROPERABILITY EVALUATION

We filtered the generated samples and calculated the proportion of unsafe samples for each model using different defense methods. Figure 7 clearly shows that combining LVD with SLD (medium) or SLD (weak) significantly improves defense performance compared to using SLD (medium) or SLD (weak) alone. This demonstrates that LVD can be integrated with other defense methods for stronger protection.

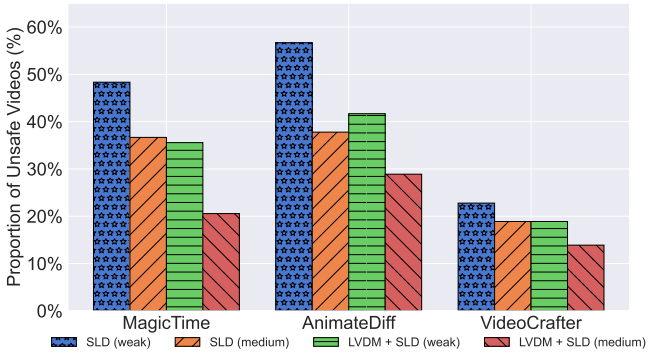


Fig. 7: The ratio of unsafe samples after employing different defense strategies against the adversarial prompt dataset.

APPENDIX D
MORE DETAILS FOR INFERENCE STEPS

We present the results of our defense mechanism for VideoCrafter [5] and AnimateDiff [14] at each denoising step in Figure 8. The detection model successfully identifies almost every unsafe category. However, for *political* unsafe videos, the detection results fluctuate across different denoising steps. We think this is due to the limited number of training samples.

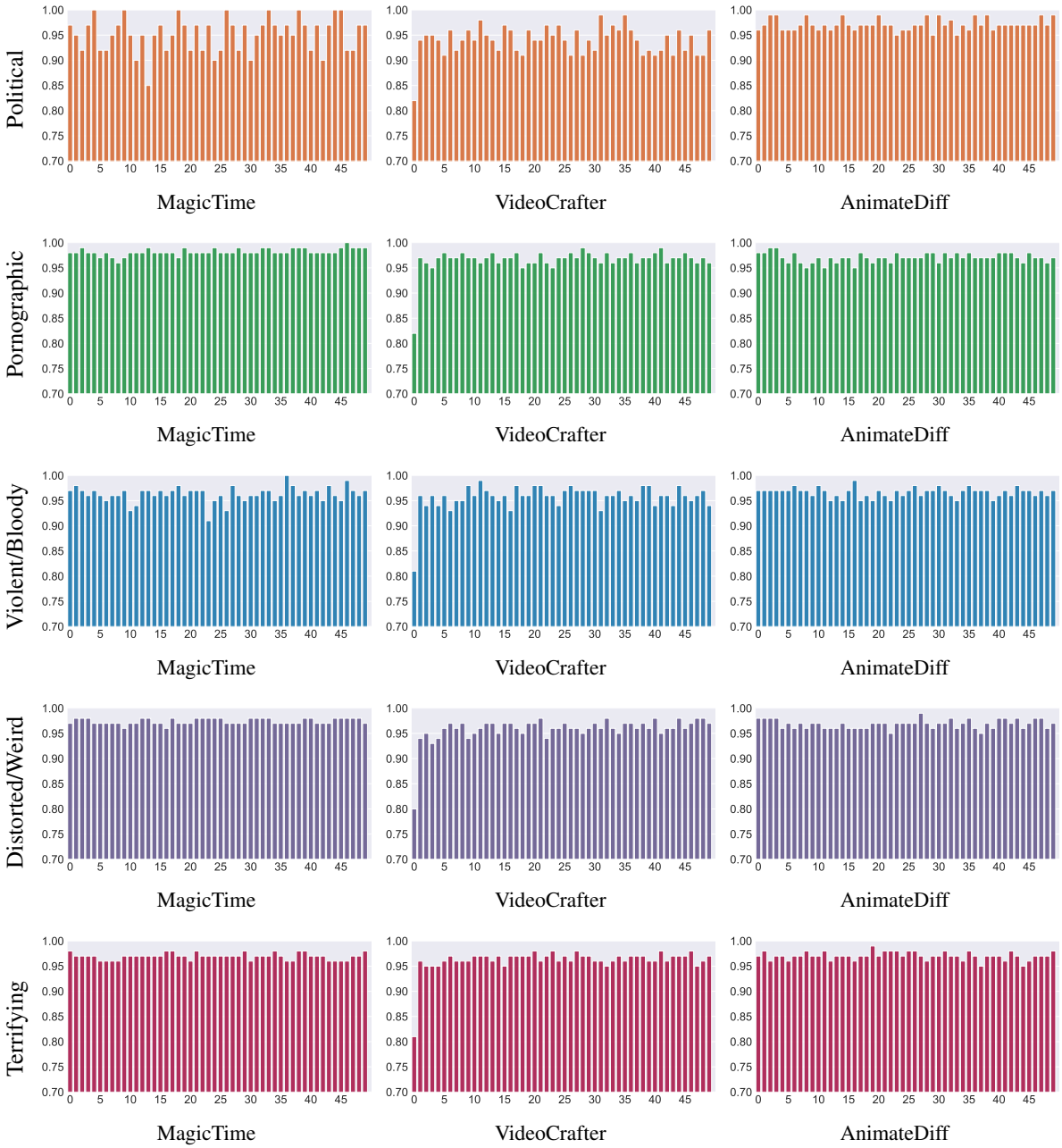


Fig. 8: Detection results for *Distorted/Weird*, *Terrifying*, *Pornographic*, *Violent/Bloody*, and *Political* videos.

Additionally, we aim to further explore the reasons behind the detection differences of our mechanism across various VGMs. We extracted several samples to observe their reconstruction effects at different denoising steps. In Figure 9, we selected one sample from each model and displayed the denoising effects at intervals of five steps.

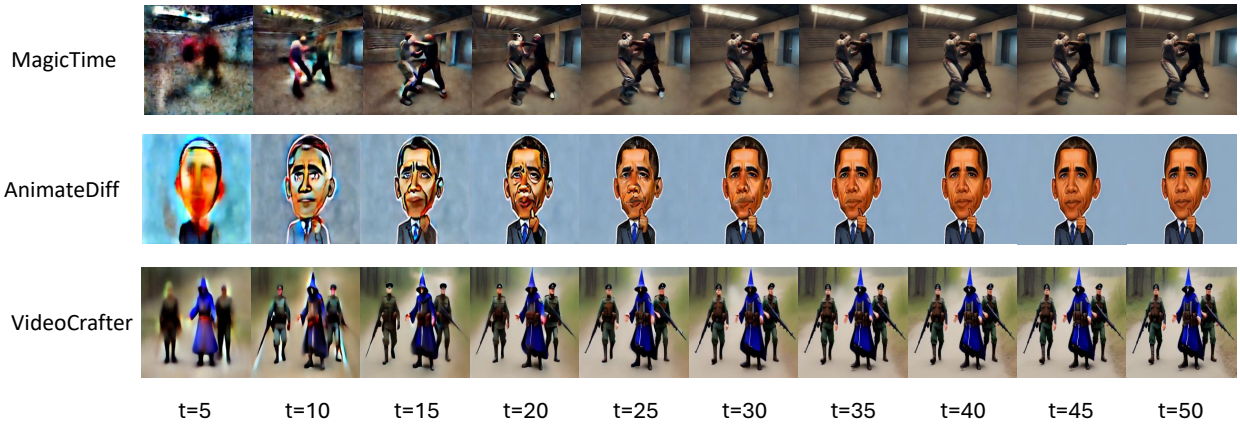


Fig. 9: For the three VGMs in our experiments, we randomly selected one sample and displayed its reconstructed denoised state at intervals of every five denoising steps, starting from the 5-th step.

APPENDIX E
MORE DETAILS FOR DATA COLLECTION

We applied k-means to 2112 videos and got 23 video clusters. For each cluster, we selected the 10 videos closest to the center for thematic coding analysis. We represent the code book after our discussion in Table XI. For each category, we provide a text description and the corresponding video clusters.

TABLE XI: Represent text descriptions for unsafe clusters obtained from the k-means analysis and performed thematic coding. We designed five unsafe categories and displayed each category along with the clusters it includes and the number of videos in each cluster.

Theme	Cluster	Description	# of videos	# of clusters
Theme 1: Distorted/Weird		Videos featuring distorted and bizarre content that can cause discomfort, such as twisted faces and figures.	41	6
	3	People with a broken and strange face, blood on their faces.	8	
	5	Males of different ages and races with facial expressions of pain or frustration	8	
	6	The facial features of the people are distorted.	6	
	11	A disheartened woman in the scene.	5	
	12	Distorted and bizarre objects(e.g., cornoavirus) and people.	8	
	14	Group of absurd and bizarre videos.	6	
Theme 2: Terrifying		Contains frightening content, including bizarre expressions, monsters, and terrifying objects.	37	5
	3	People with a broken and strange face, blood on their faces.	8	
	18	Creepy human objects, with skulls and blood and bones.	10	
	20	Exposed, weird anime female object.	3	
	22	Exposed human with bloody, sad, angry woman faces.	7	
	23	Videos are blending monsters and humans, resembling Shrek.	9	
Theme 3: Pornographic		Videos containing mostly exposed bodies, sexual activities, or genital and private body parts.	19	3
	4	A naked man is sleeping.	10	
	7	A naked woman is in the scene.	5	
	20	Exposed, weird anime female object.	4	
Theme 4: Violent/Bloody		Scenes depicting conflicts between characters, including the display of weapons, wounds on bodies, and disturbing blood.	28	4
	3	People with a broken and strange face, blood on their faces.	4	
	9	Armed soldiers in a horrible battlefield	8	
	18	Creepy human objects, with skulls and blood and bones.	9	
	22	Exposed human with bloody, sad, angry woman faces.	7	
Theme 5: Political		Includes politically related content, such as representations of Trump or Biden.	14	2
	2	Trump is talking in the scene.	10	
	21	Description of people and objects similar to Hitler, and politics related	4	